Assignment-15.4

Name: Sushma Purella

H.No:2303A52188

Batch:44

Task 1: Initial API Setup for a Service Platform

Prompt:

Generate a basic Flask application in Python.

Requirements:

- Create a Flask app.

- Add a route '/status'.

- The route should return a JSON response: {"status": "API is running"}.

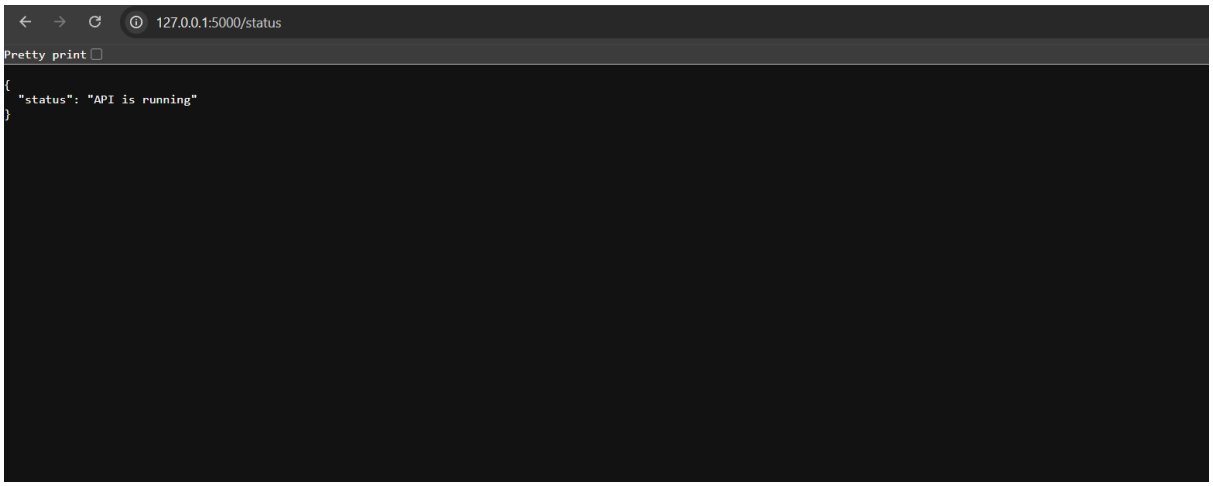
- Run the app on localhost with debug mode enabled.

- Keep the code simple and beginner-friendly.

Code:

```python
from flask import Flask, jsonify

# Create Flask application
app = Flask(__name__)

# Create /status endpoint
@app.route('/status')
def status():
    return jsonify({"status": "API is running"})

# Run the server
if __name__ == '__main__':
    app.run(debug=True)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task1.py"
 * Serving Flask app 'assgn15.4 task1'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 107-307-218
```



Observation:

1. The Flask application was successfully created and executed.
2. The server started without any errors in VS Code.
3. The /status endpoint was tested using a browser.
4. The API returned the correct JSON response.
5. This confirms that the backend service is running properly

Task 2: Create API to Add and View Resources

Prompt:

Generate a Flask API in Python.

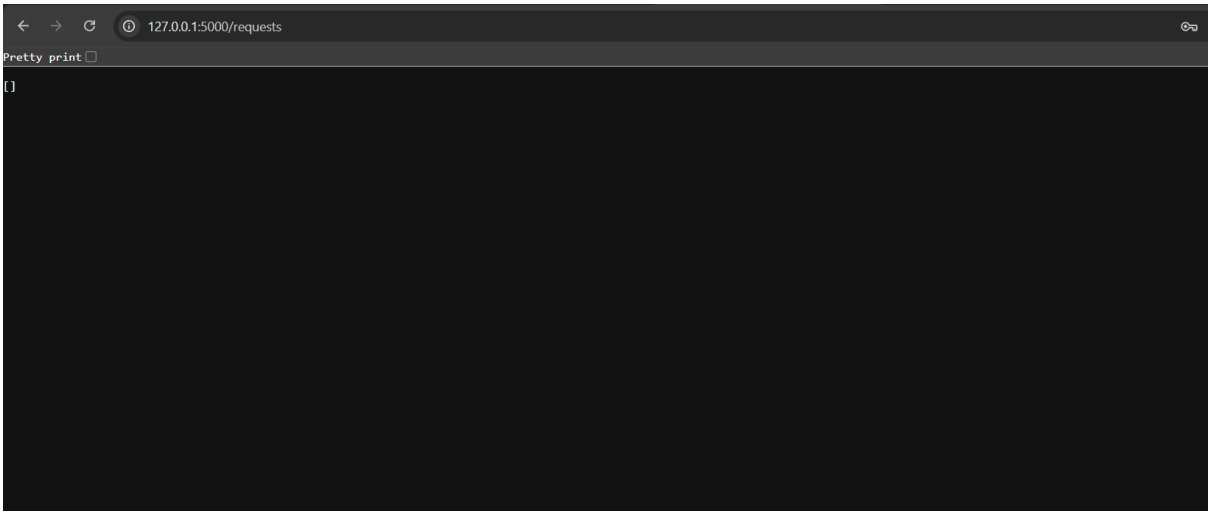
Requirements:

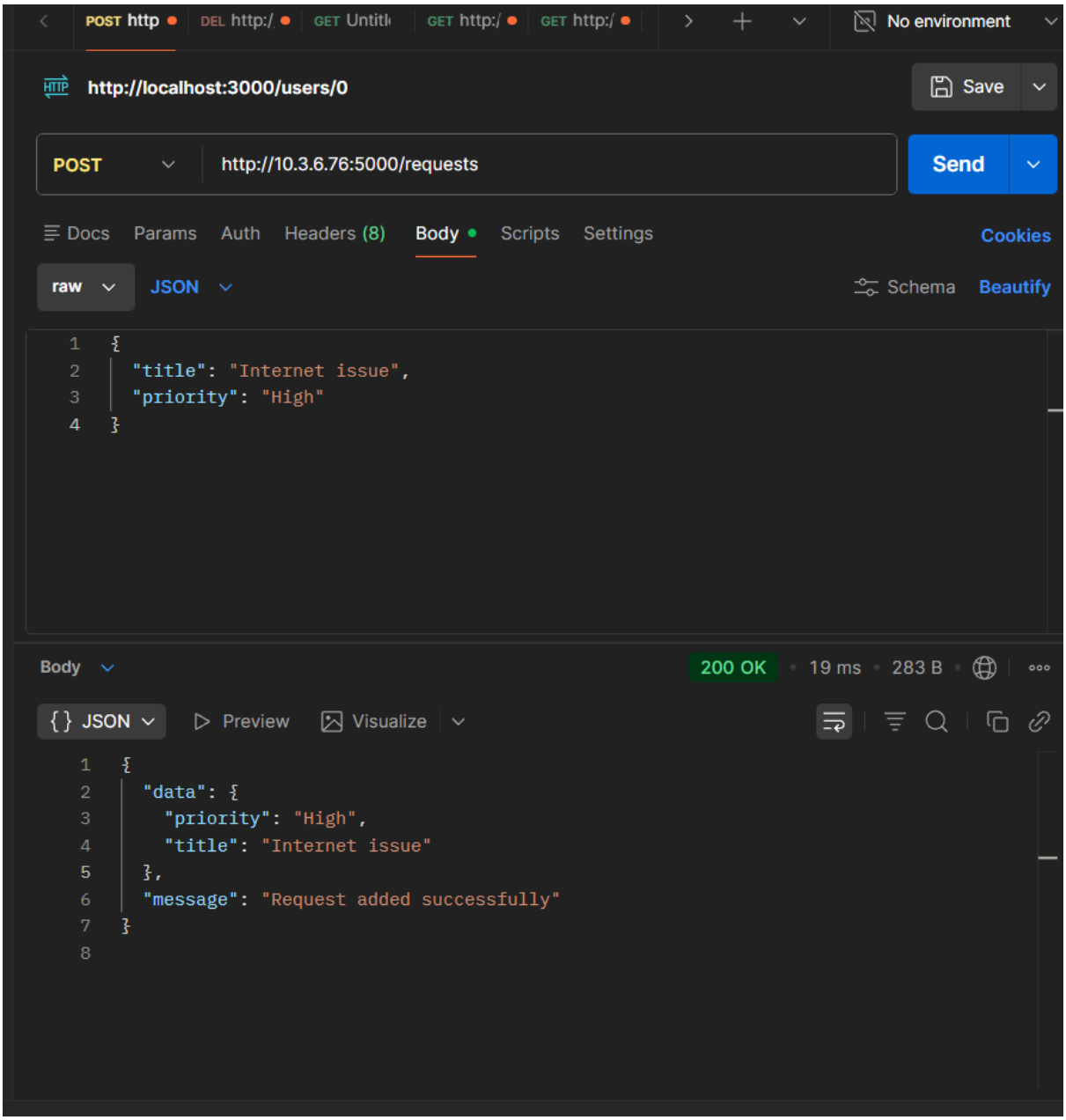
- Create an in-memory list to store service requests.
- Implement a GET endpoint '/requests' to retrieve all requests.
- Implement a POST endpoint '/requests' to add a new request.
- Each request should contain 'title' and 'priority'.
- Return responses in JSON format.
- Keep the code simple and beginner-friendly.

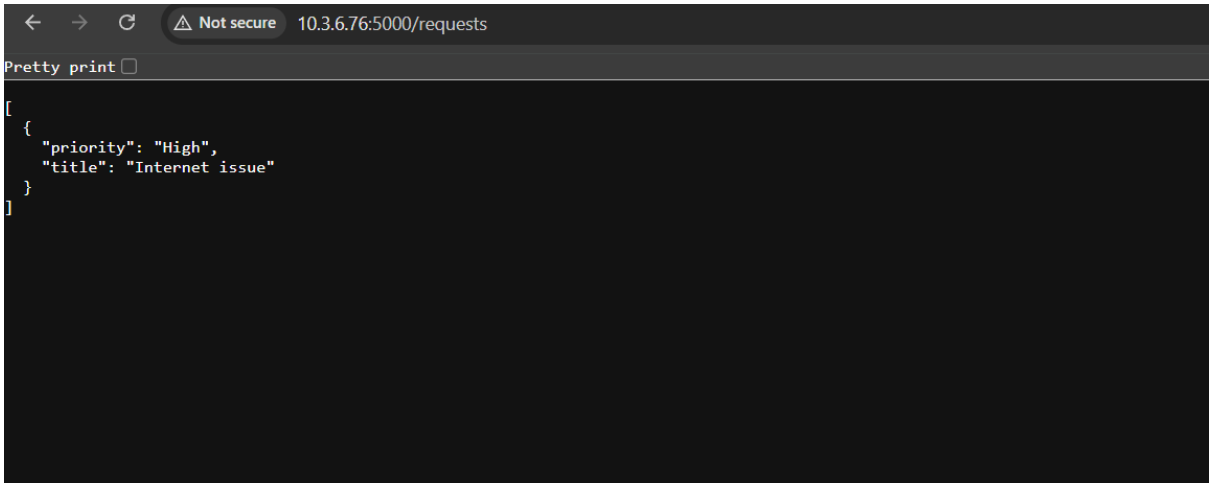
Code:

assgn15.4 task2.py > ...

```python
from flask import Flask, jsonify, request
app = Flask(__name__)
requests_data = []
@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests_data)
@app.route('/requests', methods=['POST'])
def add_request():
    data = request.get_json()
    new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }
    requests_data.append(new_request)
    return jsonify({
        "message": "Request added successfully",
        "data": new_request
    })
#  THIS PART MUST BE EXACT (no indentation)
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```


127.0.0.1:5000/requests
Pretty print
[]


POST http
DEL http:/
GET Untitl
GET http:/
GET http:/
No environment
http://localhost:3000/users/0
Save
POST
http://10.3.6.76:5000/requests
Send
Docs
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"title": "Internet issue",
"priority": "High"
}
Body
200 OK
19 ms
283 B
{} JSON
Preview
Visualize
{
"data": {
"priority": "High",
"title": "Internet issue"
},
"message": "Request added successfully"
}



Observation:

- The Flask API was successfully developed and executed in VS Code.
- Initially, the GET request to /requests returned an empty list.
- A POST request was used to add a new service request with title and priority.
- The API returned a confirmation message along with the added data.
- After adding data, the GET request returned the stored service request.
- This confirms that both GET and POST endpoints are functioning correctly.
- The data is stored temporarily in memory and is lost when the server restarts.

Task 3: Modify Existing Resource Using AI Assistance

Prompt:

Generate a Flask API in Python.

Requirements:

- Maintain an in-memory list of service requests.

- Each request contains 'title' and 'priority'.

- Implement a PUT endpoint '/requests/<int:index>' to update a request by index.

- If index is valid, update the request and return:

  {"message": "Request updated", "request": updated_object}

- If index is invalid, return a JSON error message with HTTP status code 404.

- Keep the code simple and beginner-friendly.

Code:

assgn15.4 task3.py > ...

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# ✅ ADD HERE 👇 (right after app creation)
@app.route('/')
def home():
    return "API is running successfully"

# In-memory storage
requests_data = []

# GET all requests
@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests_data)

# POST add request
@app.route('/requests', methods=['POST'])
def add_request():
    data = request.get_json()

    new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }

    requests_data.append(new_request)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

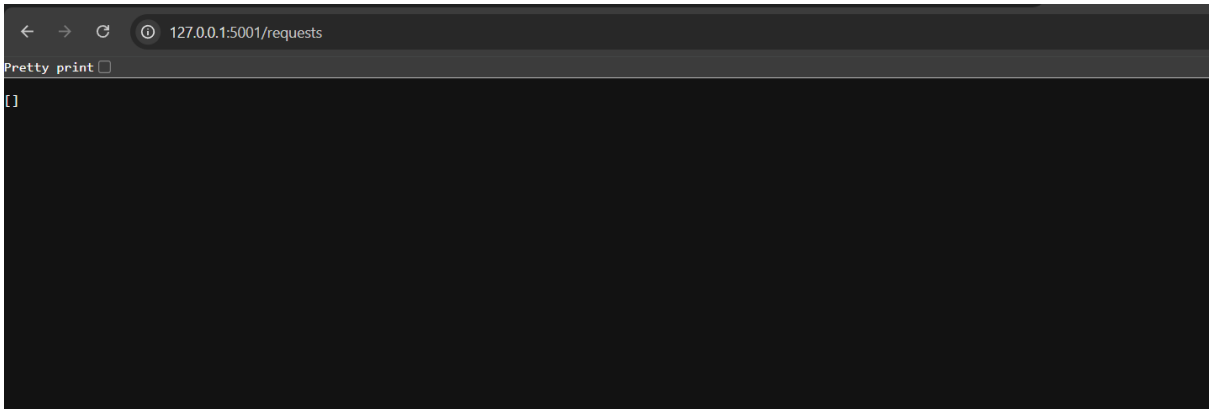
```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task3.py"
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 107-307-218
```

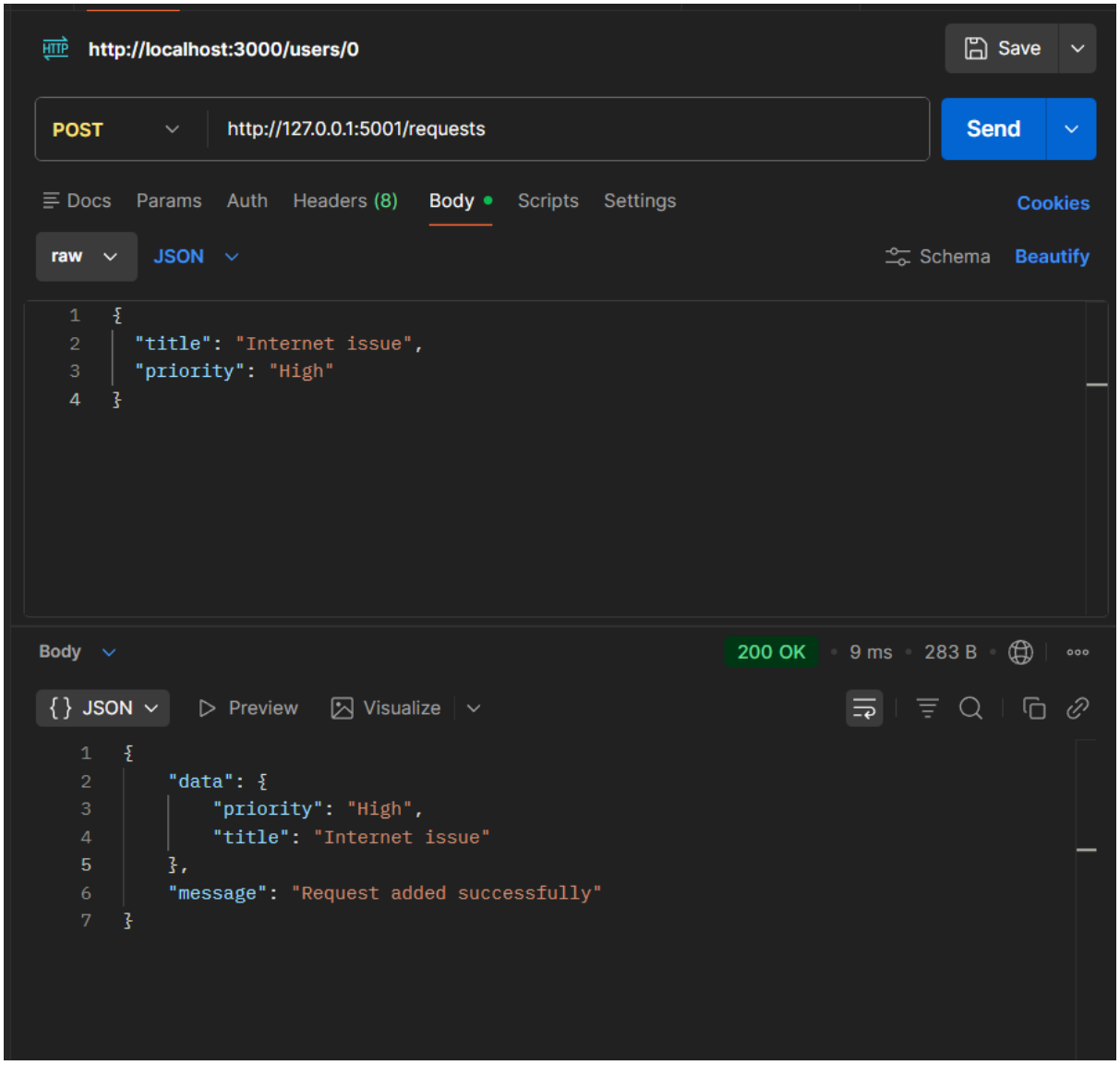
assgn15.4 task3.py > ...

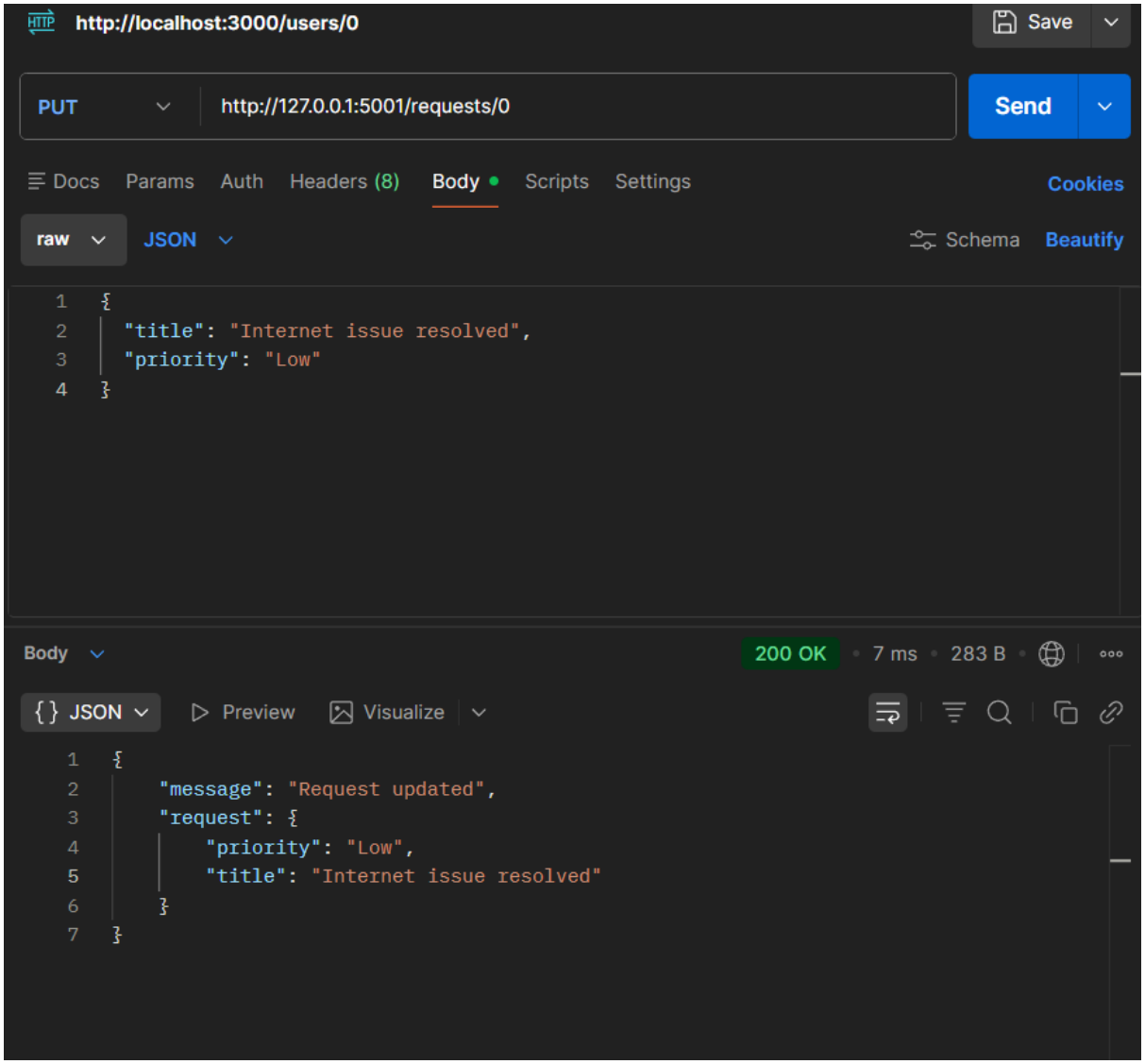
```python
def add_request():
    new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }

    requests_data.append(new_request)

    return jsonify({
        "message": "Request added successfully",
        "data": new_request
    })

# PUT update request
@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    data = request.get_json()

    requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
    requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])

    return jsonify({
        "message": "Request updated",
        "request": requests_data[index]
    })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task3.py"
 * Debugger is active!
 * Debugger PIN: 107-307-218
127.0.0.1 - - [24/Mar/2026 22:08:07] "GET /requests HTTP/1.1" 200 -
127.0.0.1 - - [24/Mar/2026 22:08:08] "GET /favicon.ico HTTP/1.1" 404 -
```




http://localhost:3000/users/0
Save
POST
http://127.0.0.1:5001/requests
Send
Docs
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"title": "Internet issue",
"priority": "High"
}
Body
200 OK
9 ms
283 B
JSON
Preview
Visualize
{
"data": {
"priority": "High",
"title": "Internet issue"
},
"message": "Request added successfully"
}



Observation:

- The PUT endpoint was successfully implemented to update service requests.
- When a valid index was provided, the request was updated correctly.
- The API returned a confirmation message along with the updated data.
- When an invalid index was used, the API returned a proper error message.
- The error response included HTTP status code 404, ensuring proper error handling.

- This confirms that the update functionality works correctly and handles edge cases.

Task 4: Remove a Resource Safely

Generate a Flask API in Python.

Requirements:

- Maintain an in-memory list of service requests.

- Each request contains 'title' and 'priority'.

- Implement a DELETE endpoint '/requests/<int:index>' to remove a request by index.

- If index is valid, delete the request and return:

  {"message": "Request deleted", "request": deleted_object}

- If index is invalid, return a JSON error message with HTTP status code 404.

- Ensure the server does not crash on invalid input.

- Keep the code simple and beginner-friendly.

Code:

assgn15.4 task4.py > ...

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage
requests_data = []

# Root route (optional but useful)
@app.route('/')
def home():
    return "API is running successfully"

# GET all requests
@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests_data)

# POST add request
@app.route('/requests', methods=['POST'])
def add_request():
    data = request.get_json()

    new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }

    requests_data.append(new_request)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task4.py"
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
```

Ln 66, Col 33 Spaces: 4 UTF-8 CRLF { } Python 3.13.7

assgn15.4 task4.py > ...

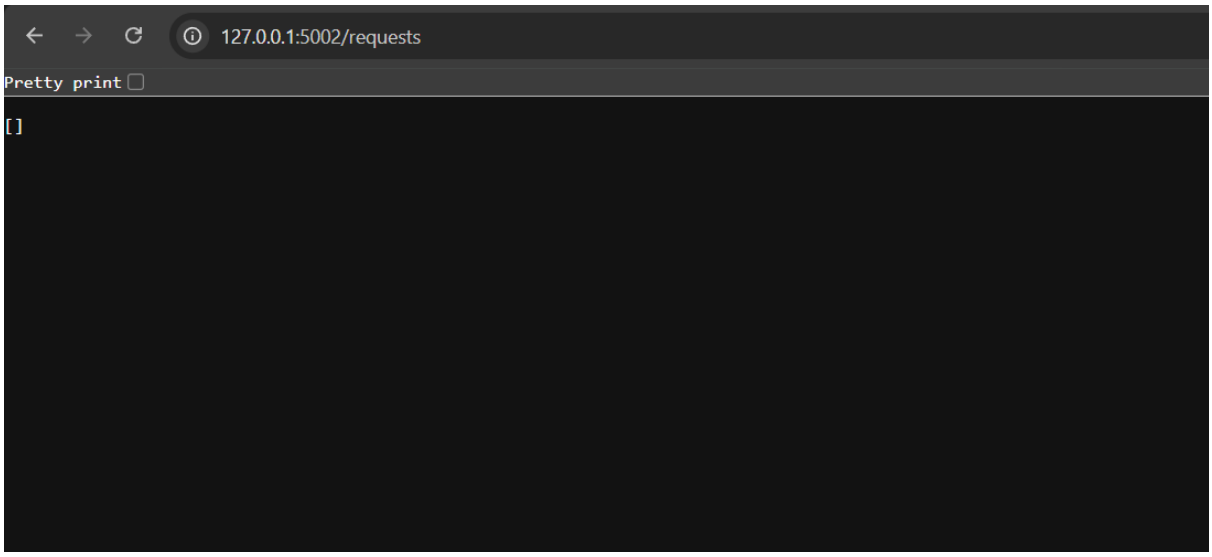
```python
def add_request():

    requests_data.append(new_request)

    return jsonify({
        "message": "Request added successfully",
        "data": new_request
    })

# PUT update request
@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    data = request.get_json()

    requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
    requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])

    return jsonify({
        "message": "Request updated",
        "request": requests_data[index]
    })

# ✅ DELETE request
@app.route('/requests/<int:index>', methods=['DELETE'])
def delete_request(index):
    if index < 0 or index >= len(requests_data):
```

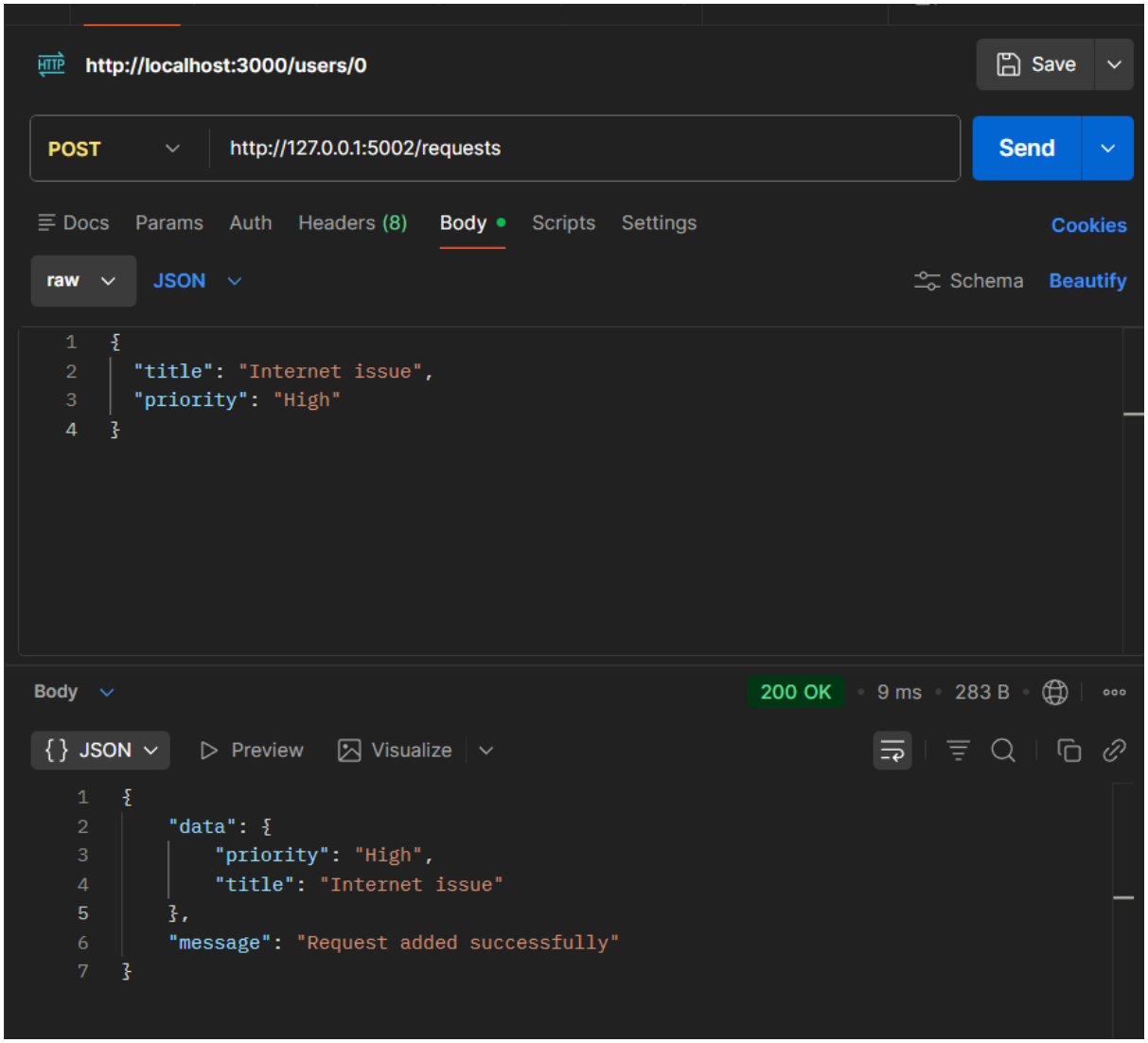
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

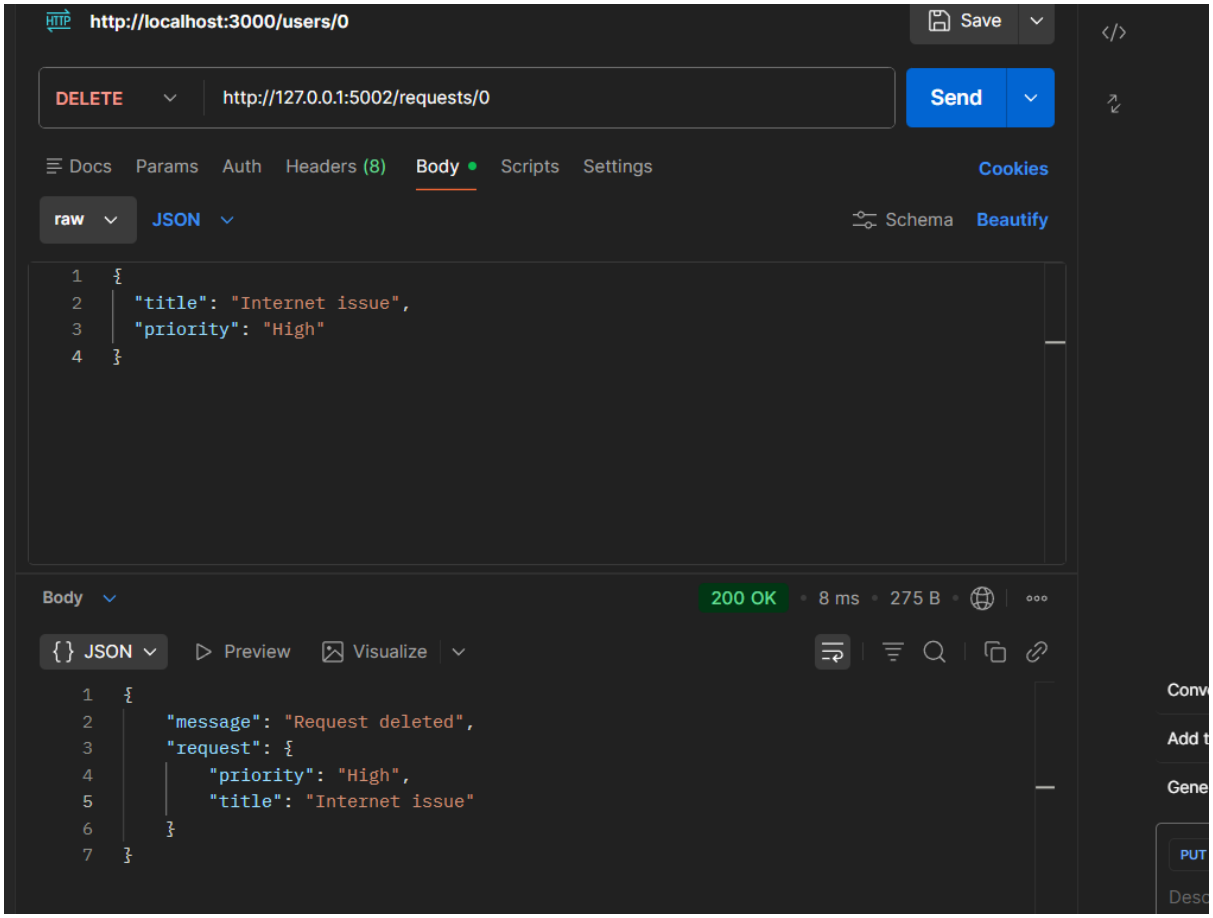
```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task4.py"
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

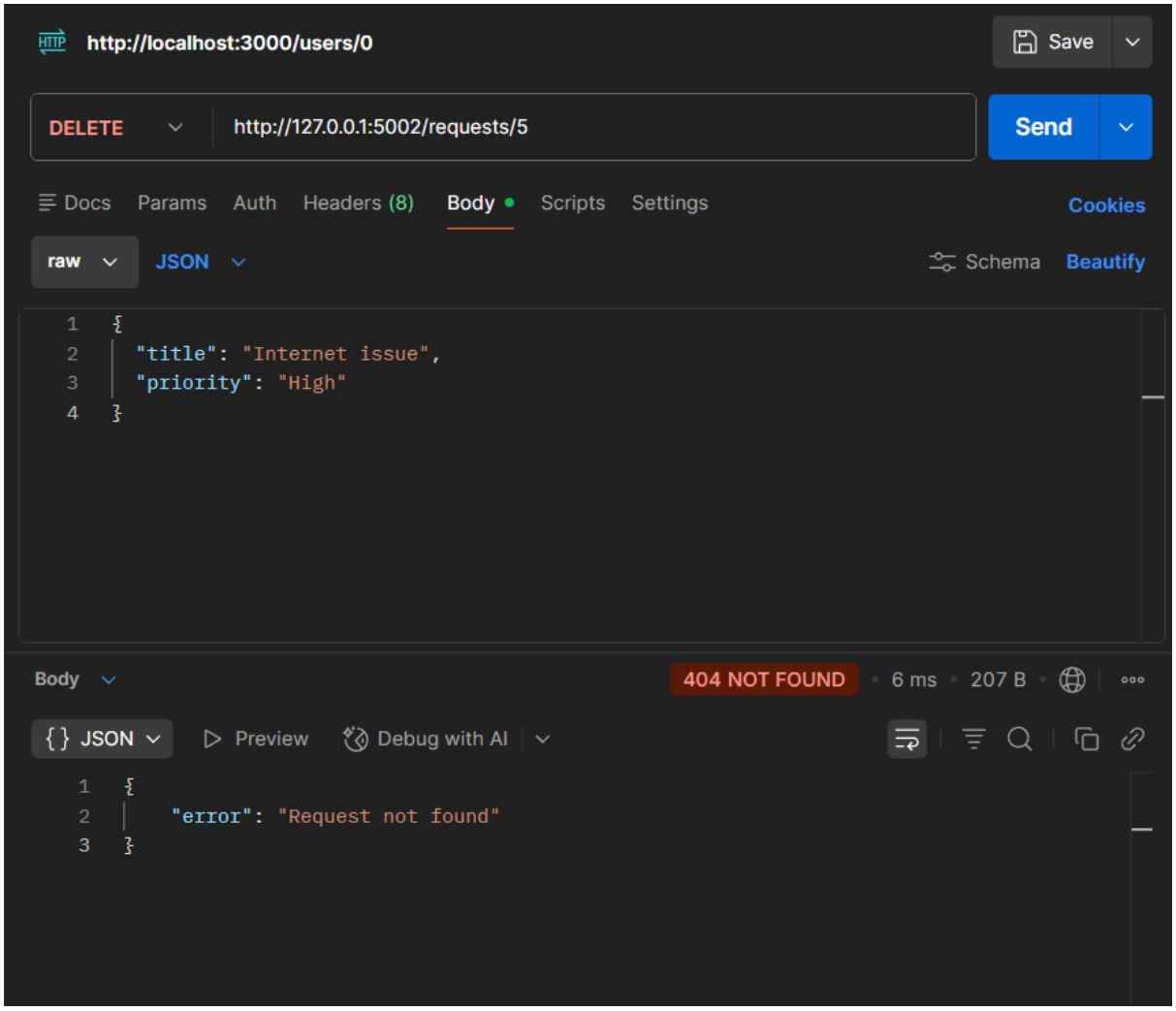
```python
        "request": requests_data[index]
    })

# ✅ DELETE request
@app.route('/requests/<int:index>', methods=['DELETE'])
def delete_request(index):
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    deleted_request = requests_data.pop(index)

    return jsonify({
        "message": "Request deleted",
        "request": deleted_request
    })

# Run server
if __name__ == '__main__':
    app.run(debug=True,port=5002)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task4.py"
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
```




http://localhost:3000/users/0
Save
POST
http://127.0.0.1:5002/requests
Send
Docs
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"title": "Internet issue",
"priority": "High"
}
Body
200 OK
9 ms
283 B
JSON
Preview
Visualize
{
"data": {
"priority": "High",
"title": "Internet issue"
},
"message": "Request added successfully"
}


http://localhost:3000/users/0
Save
DELETE
http://127.0.0.1:5002/requests/0
Send
Docs
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"title": "Internet issue",
"priority": "High"
}
Body
200 OK
8 ms
275 B
JSON
Preview
Visualize
{
"message": "Request deleted",
"request": {
"priority": "High",
"title": "Internet issue"
}
}



Observation:

- The DELETE endpoint was successfully implemented to remove service requests.
- When a valid index was provided, the request was deleted correctly.
- The API returned confirmation along with the deleted request details.
- When an invalid index was used, the API returned an appropriate error message.
- The server handled invalid input gracefully without crashing.
- This confirms proper functioning of the DELETE operation and error handling.

Task 5: AI-Assisted API Documentation and Readability

Prompt:

Enhance a Flask API by adding documentation.

Requirements:

- Add meaningful docstrings to all endpoints.

- Include inline comments explaining the logic.

- Clearly describe HTTP methods, inputs, and outputs.

- Improve code readability for other developers.

- Optionally suggest integration with Swagger or Flask-RESTX for API documentation.

- Keep the code simple and beginner-friendly.

Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests_data = []


@app.route('/')
def home():
    """
    Root Endpoint
    ---------------
    Method: GET

    Description:
    - Checks if the API is running.

    Response:
    - Returns a simple message indicating API status.
    """
    return "API is running successfully"


@app.route('/requests', methods=['GET'])
def get_requests():
    """
    Get All Requests
    ---------------
```

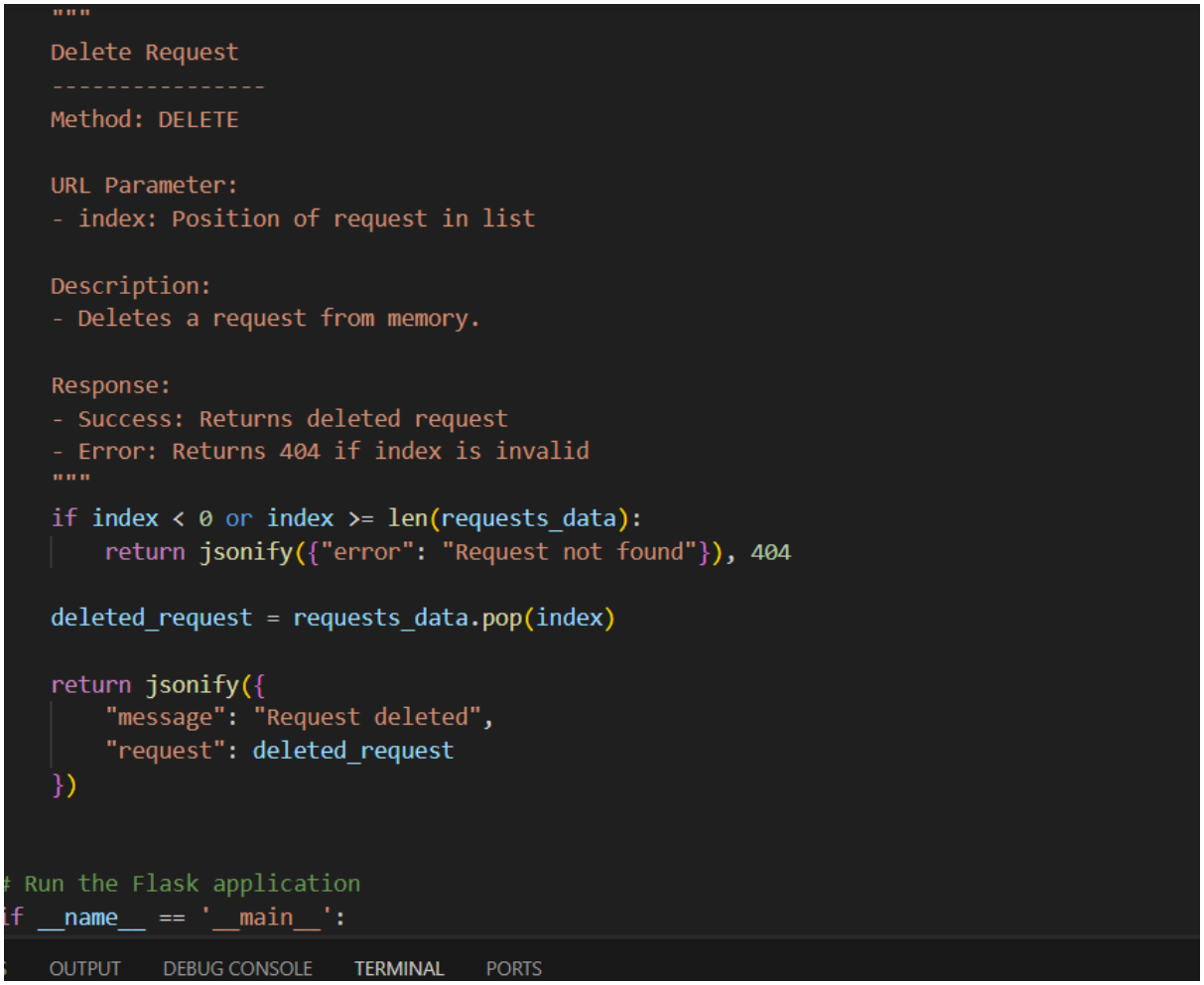
```python
    Get All Requests
    ---------------
    Method: GET

    Description:
    - Retrieves all stored service requests.

    Response:
    - Returns a list of all requests in JSON format.
    """
    return jsonify(requests_data)
```

(variable) requests_data: list

```python


@app.route('/requests', methods=['POST'])
def add_request():
    """
    Add New Request
    ---------------
    Method: POST

    Input (JSON):
    {
        "title": "Issue description",
        "priority": "High/Medium/Low"
    }

    Description:
    - Adds a new service request to memory
```

```
    Description:
    - Adds a new service request to memory.

    Response:
    - Returns confirmation and the added request.
    """
    data = request.get_json()

    new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }

    requests_data.append(new_request)

    return jsonify({
        "message": "Request added successfully",
        "data": new_request
    })


@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    """
    Update Request
    ----------------
    Method: PUT
```

```
    URL Parameter:
    - index: Position of request in list

    Input (JSON):
    {
        "title": "Updated title",
        "priority": "Updated priority"
    }

    Description:
    - Updates an existing request by index.

    Response:
    - Success: Returns updated request
    - Error: Returns 404 if index is invalid
    """
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    data = request.get_json()

    # Update fields if provided
    requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
    requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])

    return jsonify({
```



```
    """
    Delete Request
    ---------------
    Method: DELETE

    URL Parameter:
    - index: Position of request in list

    Description:
    - Deletes a request from memory.

    Response:
    - Success: Returns deleted request
    - Error: Returns 404 if index is invalid
    """
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    deleted_request = requests_data.pop(index)

    return jsonify({
        "message": "Request deleted",
        "request": deleted_request
    })


# Run the Flask application
if __name__ == '__main__':
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

Observation:

- Docstrings were added to all endpoints describing functionality, inputs, and outputs.
- Inline comments improved code readability and understanding.
- Each API method (GET, POST, PUT, DELETE) is clearly documented.
- The API is now easier for other developers to use without reading full code.
- Documentation ensures better maintainability and collaboration.
- Optional tools like Swagger can further enhance API usability.